GITHUB COPILOT · LEGACY MICROSERVICES · GKE + APIGEE

# GitHub Copilot for Legacy Microservices Analysis Guide

Analyze existing production services · GKE Kubernetes context · Apigee Basic Auth + JWT
Generate stories from code · Fix legacy issues · Build tests · Deploy safely

Copilot Chat @workspace | ☸ GKE Context | 🔑 Apigee Auth | 📝 Story Gen | 🧪 Tests | 🚀 CI/CD

Step-by-step prompts for every phase — from code scan to production GKE deployment

SETUP

# Copilot Setup — One-Time Per Service

💡 Copilot (VS Code) = inline analysis, per-file prompts, coding assistant.  Claude Code (CLI) = multi-service batch scans, full pipeline orchestration. Use both together.

1

## Install Copilot + Copilot Chat Extensions

github.copilot + github.copilot-chat in VS Code. Requires Copilot Business/Enterprise license. Sign in with GitHub org account.

2

## One VS Code workspace per microservice

Don't open all services together — Copilot context quality drops. Use File → Add Folder for shared libraries only.

3

## Install Kubernetes + Cloud Code extensions

ms-kubernetes-tools.vscode-kubernetes-tools + GoogleCloudTools.cloudcode. Copilot then understands K8s manifests and GKE context.

4

## Create .github/copilot-instructions.md per repo

Copilot reads this file automatically for every suggestion. This is how you train Copilot on YOUR service's patterns, auth model, and GKE config.

SETUP

# .github/copilot-instructions.md — The Most Important File

## What to Include

**Tech stack**
Java version, Spring Boot version, DB type

**Auth model**
Which endpoints use Apigee Basic Auth vs JWT passthrough

**GKE config**
Namespace, resource limits, ConfigMap/Secret key names

**Code patterns**
Exception format, DTO rules, test framework (JUnit 4/5)

**NEVER DO list**
Hardcoded secrets, Basic Auth in service layer, System.out

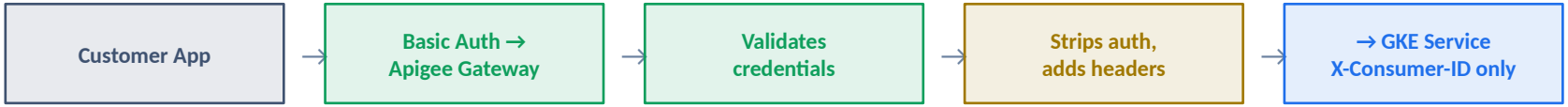
**API contracts**
Which endpoints are customer-facing and must not change

```
# copilot-instructions.md — user-service

## AUTH MODEL (critical)
# Pattern A: Customer → Apigee Basic Auth
#   Apigee validates + strips auth header
#   Service sees X-Consumer-ID header only
# Pattern B: Partner → JWT passthrough
#   Apigee validates JWT + forwards it
#   Service re-validates via JwtAuthFilter

## GKE DEPLOYMENT
# Namespace: production | staging
# Memory limit 512Mi → JVM -Xmx384m
# Config → K8s ConfigMap: user-svc-config
# Secrets → K8s Secret: user-svc-secret

## NEVER DO
# Never parse Basic Auth in service code
# Never hardcode JWT secrets or DB passwords
```

APIGEE AUTH

# Apigee + Auth Architecture — What Copilot Must Understand

## PATTERN A — Basic Auth via Apigee (most external customer APIs)




Service code sees: NO Authorization header. Reads X-Consumer-ID for customer identity. Never parse Basic Auth.

## PATTERN B — JWT Passthrough (admin / partner APIs)


Service code: JwtAuthFilter re-validates. Extracts userId, roles[], sessionId from claims. Sets MDC context.

**COPILOT RULES FOR AUTH:**

- Never generate Basic Auth parsing code in service layer
- JwtAuthFilter is the ONLY JWT validation point per service
- New endpoints must be labeled: BASIC_AUTH_VIA_APIGEE | JWT | PUBLIC
- /actuator/** always in permitAll() — K8s probes must reach it

CODE SCAN

# 4 Scan Prompts — Run in Sequence Per Service

Open Copilot Chat (Ctrl+Shift+I) · Use @workspace before each prompt · Save output to .md file in repo

## 1 Prompt 1: Service Identity & Stack Scan

- ▸ Tech stack, Java/Spring version, all endpoints
- ▸ Customer-facing endpoints + Apigee proxy routes
- ▸ Code health: TODO count, deprecated APIs, N+1 risks
- ▸ Test state: JUnit version, coverage estimate, @Ignore

→ SERVICE_PROFILE.md

## 2 Prompt 2: Apigee & Auth Flow Deep Scan

- ▸ Which endpoints receive JWT vs Basic Auth (via Apigee)
- ▸ Any endpoint incorrectly parsing Basic Auth itself?
- ▸ JwtAuthFilter coverage — are actuator endpoints excluded?
- ▸ Apigee headers (X-Consumer-ID) captured in MDC?

→ APIGEE_AUTH_MAP.md

## 3 Prompt 3: GKE Deployment Readiness

- ▸ Liveness/readiness probes configured correctly?
- ▸ JVM heap vs container memory limit aligned (75% rule)?
- ▸ Is app truly stateless (no in-memory session state)?
- ▸ ConfigMaps/Secrets — any hardcoded values found?

→ GKE_READINESS.md

## 4 Prompt 4: Inter-Service Dependencies

- ▸ All outbound HTTP: timeout configured? Circuit breaker?
- ▸ Message queue: topic names, producer/consumer, DLQ?
- ▸ Shared DB tables (blast radius concern)?
- ▸ Shared library versions — consistent across services?

→ DEPENDENCY_MAP.md

PROMPTS

# What the Scan Prompts Look Like in Copilot Chat

## Example: Prompt 2 — Apigee Auth Scan

```
@workspace #file:src/main/java

Analyze security model. We use two auth patterns:

Pattern A: Apigee handles Basic Auth,
  service receives X-Consumer-ID only
Pattern B: Apigee passes JWT through,
  service re-validates via JwtAuthFilter

Find and report:
1. Endpoints receiving JWT vs Basic Auth
2. Any endpoint parsing Basic Auth itself?
3. JwtAuthFilter excludes /actuator/**?
4. X-Consumer-ID captured in MDC?
```

## Output: APIGEE_AUTH_MAP.md

```
## APIGEE AUTH MAP

### Pattern A (Basic Auth @ Apigee)
GET /api/v1/orders
POST /api/v1/orders
GET /api/v1/users/{id}

### Pattern B (JWT Passthrough)
POST /api/v1/admin/users
DELETE /api/v1/admin/users/{id}

### 🚨 FINDINGS
UserController.java:47
Basic Auth parsed directly!
Fix: remove, use X-Consumer-ID
```

GKE

# GKE-Specific Prompts — Kubernetes & Cloud Config

## Critical GKE Rule: JVM heap must = 75% of K8s container memory limit

| Container Limit | JVM -Xmx | JVM -Xms | Flags |
|---|---|---|---|
| 256Mi | 192m | 128m | -XX:+UseContainerSupport |
| 512Mi | 384m | 256m | -XX:MaxRAMPercentage=75.0 |
| 1Gi | 768m | 512m | -Dspring.profiles.active=${PROFILE} |

### Prompt 3: GKE Manifest Review

```
@workspace #file:k8s

Verify K8s manifests for GKE:
• Probes: liveness /actuator/health/liveness
  readiness /actuator/health/readiness
• JVM heap ≤ 75% container memory limit
• All env vars from ConfigMap/Secret refs
• PodDisruptionBudget: minAvailable 1
• preStop sleep 5 (connection drain)
```

### Prompt 6: Fix K8s Manifests

```
@workspace
#file:k8s/deployment.yaml

Review and correct this deployment:
• Add missing probes
• Fix JVM -Xmx if > 75% of limit
• Replace hardcoded env values
• Add graceful termination (60s)
• Add PDB if missing
Show diff only, explain each change
```

### Prompt 7: ConfigMap/Secret Audit

```
@workspace
#file:src/main/resources

Find hardcoded values that need
externalization to K8s:
• ConfigMap: DB URLs, service URLs
• Secret: passwords, JWT secrets,
  API keys, Apigee credentials
Generate corrected application.yml
and K8s env block for deployment
```

STORY GEN

# Story Generation from Code — Two Types

## Type A — Reverse Stories

Document what code CURRENTLY does.
Baseline for regression protection.
Derived from controllers + service layer.

Prompt 8 | no new dev

## Type B — Gap Stories

What service SHOULD do but doesn't.
Security gaps, missing tests, broken code,
GKE misconfig, reliability issues.

Prompt 9 | sprint work

```
# user-service — Story Output from Copilot

TYPE A (Reverse):
US-E001 [Existing] Customer Auth via Apigee      5pt  HIGH  BASIC_AUTH  @workspace found: 3 endpoints
US-E002 [Existing] Admin User CRUD + Roles       5pt  HIGH  JWT         Uses JwtAuthFilter correctly
US-E003 [Existing] Customer Profile Self-Service 3pt  HIGH  BASIC_AUTH  Missing null check on X-Consumer-ID

TYPE B (Gap):                Priority  Touches Customer API?
US-G001 [Fix] POST /users no @Valid input validation  P1  YES  ← SECURITY
US-G002 [Fix] JVM heap 512m but K8s limit 512Mi       P1  NO   ← GKE OOM RISK
US-G003 [Fix] userId logged in plaintext (PII leak)   P1  NO   ← SECURITY
US-G004 [Add] Unit tests for UserService (0% now)     P2  NO
```

TESTS

# Test Generation Prompts — Apigee Auth Headers Included

🚨 Safety rule: Before accepting ANY Copilot test suggestion — verify it doesn't change production method signatures or HTTP contracts.

## Prompt 14: Unit Tests — JUnit 4 Style

► Match BaseServiceTest.java — JUnit 4 @RunWith, NOT @ExtendWith

► @Mock fields, @InjectMocks — no Spring context load

► Test: happy path, not-found, validation fail, downstream failure

► Mock SecurityContext if method reads current user from JWT

⚡ Use JUnit 4 (not 5) unless service already uses JUnit 5

## Prompt 15: Controller Tests + Apigee Headers

► @WebMvcTest (NOT @SpringBootTest — too slow)

► Pattern A endpoints: add .header("X-Consumer-ID","test-123")

► Pattern B endpoints: add .header("Authorization","Bearer "+jwt)

► Test 401/403 for JWT endpoints, missing X-Consumer-ID for Basic Auth

⚡ Build TestJwtBuilder helper — reuse across all controller tests

## Prompt 16: Fix Failing Tests

► Paste full mvn test output into Copilot Chat

► Root cause: H2 compat, missing Apigee headers, schema drift

► Fix the TEST not prod code (unless prod is clearly wrong)

► @Ignored tests: keep ignored, add INVESTIGATE comment + JIRA

⚡ One failing test at a time — never batch-fix blind

CHEATSHEET

# All 18 Prompts — Quick Reference Index

| | | | | |
|---|---|---|---|---|
| 1 | **Service Identity & Stack** | `@workspace` | SERVICE_PROFILE.md | First, per service |
| 2 | **Apigee & Auth Flow Scan** | `@workspace` | APIGEE_AUTH_MAP.md | After Prompt 1 |
| 3 | **GKE Deployment Readiness** | `@workspace #file:k8s` | GKE_READINESS.md | After Prompt 1 |
| 4 | **Inter-Service Dependencies** | `@workspace` | DEPENDENCY_MAP.md | After Prompt 1 |
| 5 | **Auth Gap Analysis** | `@workspace` | Security findings | Before any code change |
| 6 | **Fix K8s Manifests** | `#file:k8s/deployment.yaml` | Updated YAML | Sprint 0 |
| 7 | **ConfigMap & Secret Audit** | `#file:src/main/resources` | Externalized config | Sprint 0 |
| 8 | **Reverse Stories** | `@workspace` | Type A stories → Jira | Story gen phase |
| 9 | **Gap Stories** | `@workspace` | Type B stories → Jira | Story gen phase |
| 10–13 | **Legacy Code Fixes** | `Inline Ctrl+I or Chat` | Targeted code diffs | Sprint 0 P1 fixes |
| 14 | **Unit Tests (JUnit 4)** | `Open class + Chat` | Complete test class | Coverage phase |
| 15 | **Controller Tests + Auth** | `#file:Controller` | MockMvc tests | Coverage phase |
| 16 | **Fix Failing Tests** | `Paste mvn output` | Fixed test classes | Sprint 0 cleanup |
| 17 | **Generate K8s Manifests** | `@workspace` | 5 manifest files | Deploy prep |
| 18 | **GitHub Actions CI/CD** | `@workspace` | .github/workflows/*.yaml | CI/CD setup |

TIMELINE

# Per-Service Execution Order

## Day 1 AM — Setup copilot-instructions.md

- ► Fill in tech stack, auth model, GKE config
- ► Add NEVER DO list (no Basic Auth parsing, no hardcoded secrets)
- ► Add customer API contracts (DO NOT CHANGE list)

## Day 1 PM — Run Scan Prompts 1–4

- ► Save: SERVICE_PROFILE.md, APIGEE_AUTH_MAP.md
- ► Save: GKE_READINESS.md, DEPENDENCY_MAP.md
- ► Commit all to repo — this becomes your team's reference

## Day 2 — Security & GKE Audit

- ► Prompt 5: Auth gap analysis — fix all P1 auth findings
- ► Prompt 6+7: Fix K8s manifests + externalize config
- ► Prompt 12+13: Fix JVM memory + PII logging

## Day 3 — Story Generation

- ► Prompt 8: Reverse stories → create Type A Jira tickets
- ► Prompt 9: Gap stories → create Type B Jira tickets
- ► Human review: validate risk levels, mark P1 for Sprint 0

## Sprint 0 — P1 Gap Fixes + Test Baseline

- ► Prompt 16: Fix all currently failing tests first
- ► Prompts 10–11: Input validation, HTTP timeouts
- ► Verify: mvn test passes 100% before any new features

## Sprint 1+ — Tests + CI/CD Setup

- ► Prompts 14+15: Unit tests + controller tests with auth headers
- ► Prompts 17+18: K8s manifests + GitHub Actions pipeline
- ► Now ready for autonomous agentic feature development

# 📍 Start Tomorrow Morning

Per service — same sequence every time:

1. Create .github/copilot-instructions.md — fill auth model + GKE config
2. Open service as its own VS Code workspace (not with other services)
3. Run Prompts 1-4 in Copilot Chat → save 4 .md files to repo
4. Run Prompt 5 (auth audit) → fix all P1 security findings before anything
5. Run Prompts 8+9 → generate stories → create Jira tickets
6. Sprint 0: fix failing tests, P1 gaps only (Prompts 10-13, 16)

| 18 | 4 | 0 |
| --- | --- | --- |
| Copilot prompts | Scan docs per service | Basic Auth in service layer |